

Universidad Tecnológica Nacional
Facultad Regional Córdoba

Ejercicio Práctico N.º 1

Diseño e implementación de una Unidad de Procesamiento Harvard en SystemVerilog

Técnicas Digitales II

4R2

Gastón Grasso 401892
Franco Palombo 401910
Angelo Prieto 401012

Fecha de entrega: 9 de octubre de 2026

Índice

1 Introducción **2**
1.1 Objetivos del trabajo 2

2 Desarrollo y Verificación Experimental **3**
2.1 Arquitectura de la Unidad de Procesamiento Harvard 3
2.1.1 Descripción de los bloques constitutivos 3
2.2 Estructura y decodificación de la palabra de control 4
2.3 Síntesis lógica y esquemático RTL 4
2.4 Verificación mediante simulación 5

3 Conclusiones **9**

4 Fuentes **10**

5 Anexos: Código Fuente y Banco de Pruebas **11**
5.1 Módulo principal de la unidad Harvard (up_harvard.sv) 11
5.2 Módulo de la unidad aritmético-lógica (alu.sv) 11
5.3 Módulo del desplazador (shifter.sv) 12
5.4 Módulo del banco de registros (ram3port.sv) 13
5.5 Banco de pruebas (up_harvard_tb.sv) 13
5.6 Archivo de vectores de prueba (up_harvard_mem.txt) 14

Introducción

En este trabajo práctico se aborda el diseño, la implementación en lenguaje SystemVerilog y la validación funcional de una Unidad de Procesamiento con arquitectura tipo Harvard.

El presente trabajo concentra el desarrollo del punto 4 de la consigna: la integración jerárquica y el análisis exhaustivo de la unidad procesadora Harvard (`up_harvard`), empleando los módulos precedentes como componentes fundamentales del sistema.

Objetivos del trabajo

- Diseñar en SystemVerilog una unidad de datos modular.
- Integrar un banco de registros de tres puertos con una ALU y un Shifter.
- Sintetizar el diseño a fin de inspeccionar el esquemático RTL resultante y corroborar la correspondencia entre la descripción en hardware y la topología implementada.
- Comprobar el funcionamiento del camino de datos a través de una simulación temporal en la cual se ejecuten operaciones de carga, adición, sustracción con constantes y desplazamientos lógicos, registrando las banderas de estado generadas.

Desarrollo y Verificación Experimental

Arquitectura de la Unidad de Procesamiento Harvard

La unidad de procesamiento implementada adopta una topología basada en la arquitectura Harvard, en la cual el bus de datos y el bus de direcciones encuentran físicamente desacoplados. En este esquema, las consignas de operación llegan a través del bus de control `regCTRL`, mientras que los datos externos ingresan por las entradas independientes `dIN` y `cIN`. A su vez, las salidas del circuito proporcionan el dato procesado en `dOUT`, la dirección correspondiente en `aOUT` y el registro de estado o banderas en `sOUT`.

El diseño fue concebido de manera genérica en SystemVerilog mediante parámetros configurables:

- N: ancho de palabra del bus de datos (en la presente implementación se fijó en $N = 4$ bits).
- M: cantidad de bits de direccionamiento del banco de registros (fijado en $M = 2$ bits, lo que permite direccionar hasta $2^M = 4$ registros internos, denominados R_0, R_1, R_2 y R_3).

Descripción de los bloques constitutivos

La unidad procesadora se estructura internamente a partir de cuatro componentes principales y tres etapas de multiplexación:

1. Banco de registros (`ram3port`): constituye la memoria interna del camino de datos. Dispone de dos puertos de lectura asíncronos direccionados por las señales `aAddress` (`a1`) y `bAddress` (`a2`), cuyas salidas entregan respectivamente los buses `aBus` (`d1`) y `bBus` (`d2`). La lectura es puramente combinacional. Dispone asimismo de un puerto de escritura direccionado por `dAddress` (`a3`), el cual captura el valor presente en `dBus` (`d3`) durante el flanco ascendente de la señal de reloj (`clk`) únicamente si la señal `loadEnable` (`we3`) se encuentra en nivel alto. **NOTA:** Se modificó el archivo original provisto por la catedra con el objetivo de poder unificar la parametrizacion con los mismos nombres de parametros. Solo se cambió el parametro M por el N y viceversa.

2. Multiplexor de entrada B (`MUX B`): selecciona la fuente del operando B entre el valor leído del registro en `bBus` y una constante externa provista por `cIN`. La selección está gobernada por la señal `constInEnable`. La salida de este multiplexor conforma directamente la señal `dOUT`, que actúa como operando para las unidades funcionales.

3. Unidad aritmético-lógica (`alu`): procesa los operandos provenientes de `aOUT` (conectado a `aBus`) y `dOUT` bajo el control del vector `aluControl` de 4 bits. Permite ejecutar transferencias, incrementos, decrementos, adiciones y sustracciones mediante aritmética en complemento a dos, así como funciones lógicas a nivel de bits (AND, OR, XOR y

negación). Entrega el resultado de cálculo en `aluOut` y las banderas aritméticas en `ALUFlags` hacia `sOUT`, correspondientes a sobreflujo (V), signo negativo (N), cero (Z) y acarreo (C).

4. Unidad de corrimiento (`shifter`): toma como entrada el operando `dOUT` y aplica operaciones de desplazamiento lógico a la izquierda, corrimiento a la derecha con extensión de signo, rotaciones y puesta a cero, gobernada por el código `shifterControl` de 3 bits. Su salida se rotula como `shifterOut`.

5. Multiplexor de unidad funcional (`MUX FU`): arbitra la salida entre la ALU (`aluOut`) y el desplazador (`shifterOut`) en función del bit de control `aluEnable`. El dato seleccionado se propaga hacia la señal interna `functionUnitOut`.

6. Multiplexor de retorno D (`MUX D`): determina el origen del bus de escritura `dBus`, conmutando entre el dato externo entrante por `dIN` y el resultado producido por la unidad funcional en `functionUnitOut`. Dicha conmutación responde al estado de la señal `dataInEnable`. El bus resultante `dBus` cierra el lazo de realimentación hacia el puerto de entrada del banco de registros.

Estructura y decodificación de la palabra de control

La palabra de control `regCTRL` sintetiza todas las directivas de control del camino de datos en un vector de $3 \times M + 3 + 5 = 14$ bits.

En la figura 2.1 se ilustra la distribución de los campos dentro de `regCTRL`, mientras que en la tabla 2.1 se detalla su asignación a las variables internas del diseño y sus destinos en el hardware.

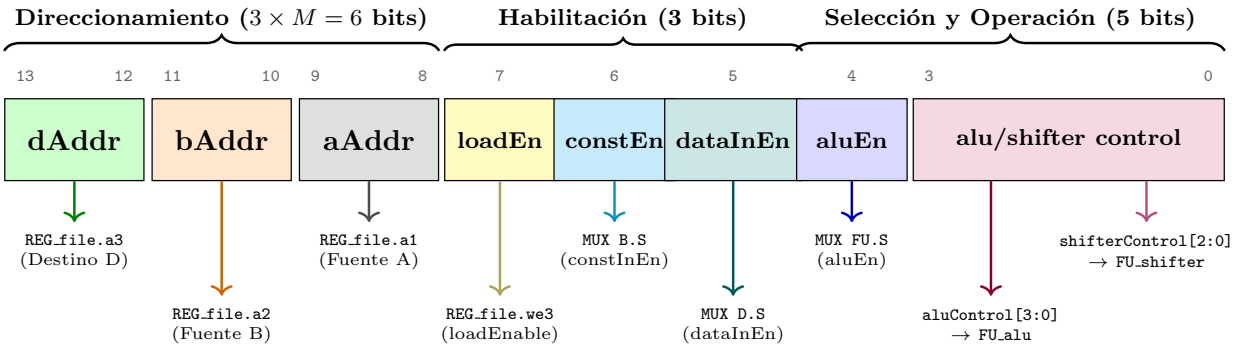


Figura 2.1: Estructura de partición de la palabra de control `regCTRL[13:0]`.

Síntesis lógica y esquemático RTL

Para corroborar la topología del circuito y examinar la distribución efectiva de las señales de control hacia cada instancia y multiplexor, se llevó a cabo la síntesis lógica.

Cuadro 2.1: Mapeo de la palabra de control en las variables internas de la unidad.

Rango de bits	Variable interna	Ancho	Función principal	Módulo de destino
[13:12]	dAddress	2 bits	Dirección de registro destino	REG_file.a3
[11:10]	bAddress	2 bits	Dirección de registro fuente B	REG_file.a2
[9:8]	aAddress	2 bits	Dirección de registro fuente A	REG_file.a1
[7]	loadEnable	1 bit	Habilitación de escritura	REG_file.we3
[6]	constInEnable	1 bit	Selección de constante vs registro	MUX B.S
[5]	dataInEnable	1 bit	Selección de dato externo vs cálculo	MUX D.S
[4]	aluEnable	1 bit	Selección ALU vs Desplazador	MUX FU.S
[3:0]	aluControl	4 bits	Operación de la ALU	FU_alu.ALUControl
[2:0]	shifterControl	3 bits	Operación del desplazador	FU_shifter.S

En la figura 2.2 se expone el diagrama RTL generado. En él se aprecia de manera directa cómo la palabra de control regCTRL se descompone en sus respectivos rebanados de bits. Desde dichos nodos, las líneas de control se propagan de forma unívoca:

- Las variables de direccionamiento aAddress, bAddress y dAddress, junto con la señal de habilitación de escritura loadEnable, ingresan al banco de registros REG_file.
- La variable aluControl se conecta directamente a la entrada ALUControl de la instancia FU_alu.
- La variable shifterControl ingresa a la entrada de selección S de la unidad FU_shifter.
- Las variables constInEnable, aluEnable y dataInEnable conmutan las entradas de control S de los multiplexores MUX B, MUX FU y MUX D, respectivamente.
- Se visualiza con total claridad el lazo cerrado de realimentación: el bus dBus, generado por la salida del multiplexor MUX D, regresa hacia la entrada de datos de escritura d3 del banco de registros.

Verificación mediante simulación

La verificación del camino de datos se efectuó a través del banco de pruebas. Cada vector del banco de prueba posee una longitud de 22 bits, estructurado como la concatenación {regCTRL, cIN, dIN}. El período del reloj de prueba se fijó en 20 ns.

El plan de prueba abarca una secuencia de 8 ciclos de reloj (T_0 a T_7) orientados a validar la carga inicial, dos operaciones aritméticas fundamentales, una operación de corrimiento y las lecturas de verificación correspondientes:

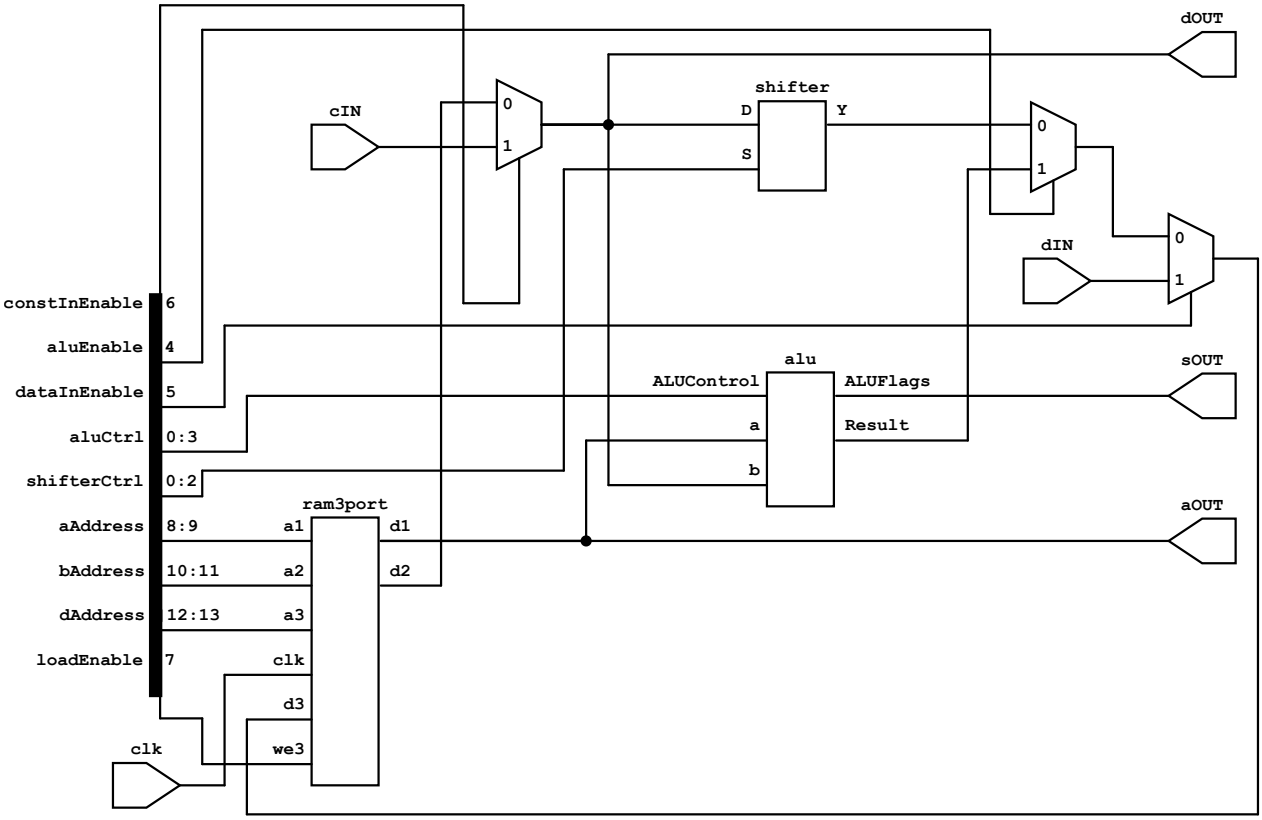


Figura 2.2: Esquemático RTL del módulo `up_harvard` sintetizado con Yosys.

1. Ciclo T_0: Carga inicial en el registro R_0. Se aplica `dIN = 5`, con `dAddress = 0`, `loadEnable = 1` y `dataInEnable = 1`. En el flanco ascendente de reloj, el valor 5 queda almacenado en R_0.

2. Ciclo T_1: Carga inicial en el registro R_1. Se aplica `dIN = 3`, con `dAddress = 1`, `loadEnable = 1` y `dataInEnable = 1`. En el flanco ascendente de reloj, el valor 3 queda almacenado en R_1.

3. Ciclo T_2: **Primera operación aritmética (Suma)**. Se ejecuta la adición $R_2 \leftarrow R_0 + R_1$. Los puertos de lectura seleccionan R_0 (`aAddress = 0`) y R_1 (`bAddress = 1`), entregando `aBus` $= 5$ y `bBus` $= 3$. Con `constInEnable = 0`, la entrada B de la ALU recibe `dOUT` $= 3$. Se configura `aluEnable = 1` y `aluControl = 0010b` (operación $A+B$). La ALU genera `aluOut` $= 5+3 = 8$. Con `dataInEnable = 0`, este resultado se transfiere a `dBus` $= 8$ y se escribe en R_2 (`dAddress = 2`, `loadEnable = 1`) en el flanco de subida siguiente. Las banderas generadas en `sOUT` son `0100b`, indicando la activación de la bandera de signo negativo ($N = 1$), debido a que en representación con signo de 4 bits el patrón `1000b` posee su bit más significativo en 1.

4. Ciclo T_3: Verificación de lectura de la suma. Se configura `bAddress = 2` con `constInEnable = 0` y `loadEnable = 0`. La salida `dOUT` refleja de inmediato el contenido de R_2 (8), confirmando que el valor fue almacenado correctamente.

5. Ciclo T_4: **Segunda operación aritmética (Resta)**. Se ejecuta la sustracción con operando constante $R_2 \leftarrow R_2 - cIN$, empleando $cIN = 2$ (`0010b`). Se direcciona `aAddress = 2` ($R_2 = 8$) y se activa `constInEnable = 1`, de modo que `dOUT` $= 2$. El control de la ALU se establece en `aluControl = 0101b`, correspondiente a la operación aritmética de sustracción $A+ \sim B + 1$. El resultado producido es `aluOut` $= 8 - 2 = 6$ (`0110b`). Al estar habilitada la escritura sobre R_2 (`dAddress = 2`, `loadEnable = 1`), el nuevo valor 6 se registra en el flanco de subida. Las banderas resultantes son `0001b`, encendiendo la bandera de acarreo ($C = 1$), lo cual certifica que no existió condición de préstamo negativo en la resta.

6. Ciclo T_5: Verificación de lectura de la resta. Se direcciona nuevamente R_2 hacia el bus B (`bAddress = 2`) con `constInEnable = 0`. La salida `dOUT` muestra el valor 6.

7. Ciclo T_6: Operación de corrimiento a la izquierda. Se aplica un desplazamiento lógico sobre R_2: $R_2 \leftarrow R_2 \ll 1$. Con `bAddress = 2` y `constInEnable = 0`, la unidad de corrimiento recibe el valor 6 por `dOUT`. Se configura `shifterControl = 001b` (SHL) y se desactiva la ALU (`aluEnable = 0`), permitiendo el paso de `shifterOut` a través de `MUX FU`. El valor resultante es `fuOut` $= 6 \ll 1 = 12$ (`1100b`), el cual se escribe en R_2 al flanco de subida.

8. Ciclo T_7: Verificación final de corrimiento. Se lee el registro R_2 en la salida `dOUT`, comprobando el valor final 12 (`1100b`).

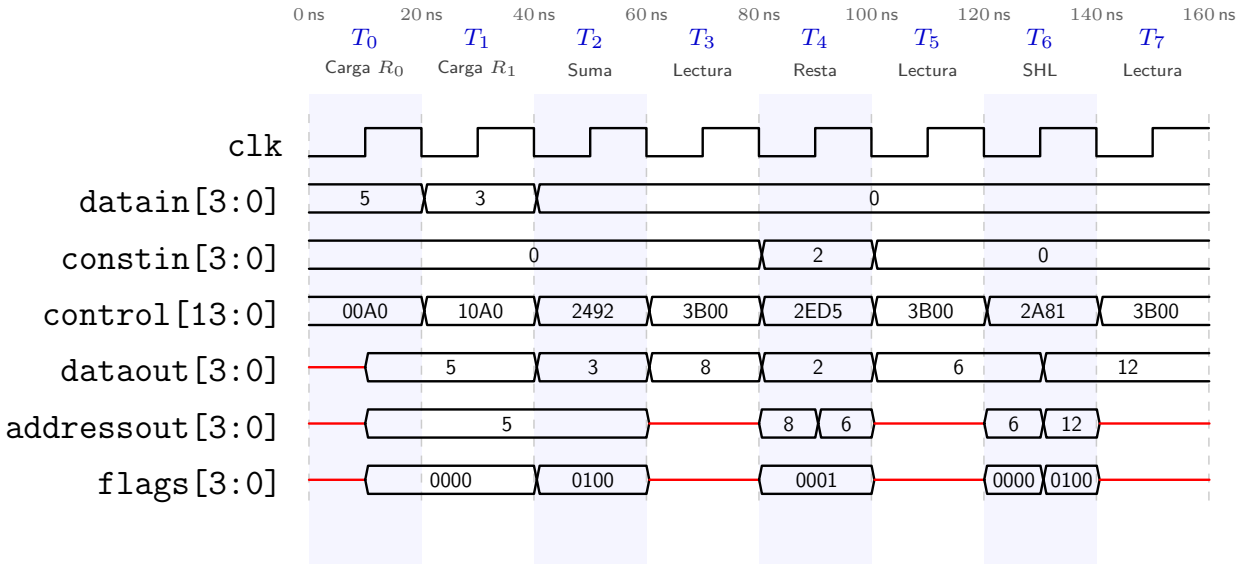


Figura 2.3: Cronograma de señales de la simulación temporal en el banco de pruebas.

En la tabla 2.2 se resume la correlación entre las palabras de control aplicadas, las señales internas y los resultados verificados en el camino de datos durante cada ciclo de prueba.

Cuadro 2.2: Resumen de la secuencia de prueba y resultados de simulación.

Ciclo	Tiempo [ns]	Operación ejecutada	`regCTRL`	`dIN`	`cIN`	`dOUT`	`dBus`	Banderas
T_0	$0-20$	Carga $R_0 \leftarrow 5$	`00A0h`	5	0	–	5	–
T_1	$20-40$	Carga $R_1 \leftarrow 3$	`04A0h`	3	0	–	3	–
T_2	$40-60$	Suma: $R_2 \leftarrow R_0 + R_1$	`2492h`	0	0	3	8	`0100b` ($N=1$)
T_3	$60-80$	Lectura R_2	`3B00h`	0	0	8	–	–
T_4	$80-100$	Resta: $R_2 \leftarrow R_2 - cIN$	`2ED5h`	0	2	2	6	`0001b` ($C=1$)
T_5	$100-120$	Lectura R_2	`3B00h`	0	0	6	–	–
T_6	$120-140$	Corrimiento: $R_2 \leftarrow R_2 \ll 1$	`2AA1h`	0	0	6	12	`0100b` ($N=1$)
T_7	$140-160$	Lectura R_2	`3B00h`	0	0	12	–	–

Conclusiones

La implementación y verificación de la unidad de procesamiento con arquitectura Harvard desarrollada en este trabajo práctico dio resultados coherentes con los requerimientos teóricos planteados para el diseño de procesadores.

En primer término, la organización jerárquica del camino de datos permitió desacoplar con éxito la memoria interna (banco de registros de tres puertos) de las unidades aritméticas y lógicas de procesamiento. La utilización de puertos de lectura puramente combinacionales y un único puerto de escritura síncrono facilitó la realización de ciclos de cálculo en los cuales los operandos se leen, se operan en la ALU o en el desplazador, y el resultado queda disponible en el bus de escritura dentro del mismo período de reloj, efectuándose la captura en memoria al presentarse el flanco activo.

En segundo lugar, se validó la eficacia de la descomposición de la palabra de control (`regCTRL`). Esta práctica resultó indispensable para verificar visualmente en el esquemático RTL cómo la palabra de control ingresa y bifurca sus campos hacia las instancias de procesamiento y los multiplexores de selección.

En tercer lugar, la simulación temporal en el banco de pruebas confirmó la ejecución exacta de las operaciones solicitadas:

- La adición entre los valores precargados en R_0 (5) y R_1 (3) produjo el resultado 8 y activó de manera correcta la bandera de signo negativo ($N = 1$), debido a la convención de signo en el bit 3.

- La sustracción entre el acumulador R_2 (8) y el dato constante externo cIN (2) a través de la vía de entrada alternativa del multiplexor B entregó el resultado 6 (`0110b`) con activación de la bandera de acarreo ($C = 1$), ratificando la adecuada implementación del complemento a dos ($A+ \sim B + 1$).

- La posterior operación de desplazamiento lógico a la izquierda duplicó el valor almacenado en R_2 alcanzando 12 (`1100b`), verificando la correcta selectividad del multiplexor de la unidad funcional.

En síntesis, la experiencia permitió comprobar de manera analítica y experimental el funcionamiento coordinado de una unidad de procesamiento Harvard, sentando las bases de hardware sobre las cuales se diseñan las unidades de control y los secuenciadores de instrucciones en arquitecturas computacionales.

Fuentes

[MKM15] M. Morris Mano, Charles R. Kime y Tom Martin. *Logic and Computer Design Fundamentals*. 5ª ed. Boston, MA, USA: Pearson, 2015.

[Iee] *IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language*. IEEE Std 1800-2017. IEEE. 2017. DOI: 10.1109/IEEESTD.2018.8299595.

[PH20] David A. Patterson y John L. Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. 6ª ed. San Francisco, CA, USA: Morgan Kaufmann, 2020.

[HH21] David Money Harris y Sarah L. Harris. *Digital Design and Computer Architecture: RISC-V Edition*. Cambridge, MA, USA: Morgan Kaufmann, 2021.

[Wol+24] Claire Wolf et al. *Yosys Open SYnthesis Suite Manual*. YosysHQ GmbH. 2024. URL: https://yosyshq.net/yosys/.

Anexos: Código Fuente y Banco de Pruebas

En este anexo se detallan los códigos fuente completos en SystemVerilog utilizados para la síntesis y simulación de la unidad procesadora con arquitectura Harvard, junto con el archivo de vectores del banco de pruebas.

Módulo principal de la unidad Harvard (up_harvard.sv)

```
module up_harvard #(parameter M = 2, N = 4)
                   (input logic clk,
                    input logic [(N-1):0] dIN, cIN, // dIN: Data In, cIN: Constant In
                    input logic [(3*M+3+5-1):0] regCTRL,
                    output logic [(N-1):0] dOUT, aOUT, // dOUT: Data Out, aOUT: Address Out
                    output logic [3:0] sOUT); // sOUT: Status Out

   logic loadEnable, constInEnable, dataInEnable, aluEnable;
   logic [(M-1):0] aAddress, bAddress, dAddress;
   logic [(N-1):0] aBus, bBus, dBus, aluOut, shifterOut, functionUnitOut;
   logic [3:0] aluControl;
   logic [2:0] shifterControl;
   assign dAddress = regCTRL[(8+3*M-1):(8+2*M)];
   assign bAddress = regCTRL[(8+2*M-1):(8+M)];
   assign aAddress = regCTRL[(8+M-1):8];
   assign loadEnable = regCTRL[7];
   assign constInEnable = regCTRL[6];
   assign dataInEnable = regCTRL[5];
   assign aluEnable = regCTRL[4];
   assign aluControl = regCTRL[3:0];
   assign shifterControl = regCTRL[2:0];

   ram3port #(.N(N), .M(M)) REG_file (clk, loadEnable, aAddress, bAddress, dAddress, aBus,
    bBus, dBus);
   assign aOUT = aBus;
   assign dOUT = constInEnable ? cIN
                               : bBus;

   alu #(.N(N)) FU_alu (aOUT, dOUT, aluControl, aluOut, sOUT);
   shifter #(.N(N)) FU_shifter (dOUT, shifterControl, shifterOut);

   assign functionUnitOut = aluEnable ? aluOut
                              : shifterOut;
   assign dBus = dataInEnable ? dIN
                              : functionUnitOut;

endmodule
```

Listing 5.1: Descripción RTL de la unidad de procesamiento Harvard (`up_harvard.sv`).

Módulo de la unidad aritmético-lógica (alu.sv)

```
module alu #(parameter N = 4)
          (input logic [N-1:0] a, b,
           input logic [3:0] ALUControl,
           output logic [N-1:0] Result,
           output logic [3:0] ALUFlags);
```

```
6
7   logic neg, zero, acarreo, overflow;
8   logic [N-1:0] y, sum;
9   logic cin, cout;
10 // ------------------------------------------------
11 //|_selección_|_  entrada _|_   F = A + y + cin   _|
12 //|_ S1  S0  _|_    y     _|_   cin=0     cin=1   _|
13 //|   0   0   | solo ceros |    F=A       F=A+1    |
14 //|   0   1   |     B      |    F=A+B     F=A+B+1  |
15 //|   1   0   |    /B      |    F=A+/B    F=A+/B+1 |
16 //|_  1   1  _|_solo unos _|_   F=A-1     F=A     _|
17
18  assign cin = ALUControl[0];
19  assign y = ALUControl[2] ? (ALUControl[1] ? {N {1'b1}}
20                                            : ~b )
21                           : (ALUControl[1] ? b
22                                            : {N {1'b0}} );
23  assign {cout,sum} = a + y + cin;
24
25  always_comb
26    casex (ALUControl[3:1])
27      3'b0??: Result = sum;
28      3'b100: Result = a & b;
29      3'b101: Result = a | b;
30      3'b110: Result = a ^ b;
31      3'b111: Result = ~a;
32    endcase
33
34  assign neg = Result[N-1];
35  assign zero = (Result == {N {1'b0}});
36  assign acarreo = (ALUControl[3] == 1'b0) & cout;
37  assign overflow = (ALUControl[3] == 1'b0) &
38                    (~a[N-1] & ~y[N-1] & sum[N-1] +
39                      a[N-1] &  y[N-1] & ~sum[N-1]);
40  assign ALUFlags = {overflow, neg, zero, acarreo };
41 endmodule
```

Listing 5.2: Implementación de la unidad aritmético-lógica (`alu.sv`).

Módulo del desplazador (shifter.sv)

```
1  module shifter #(parameter N = 4)
2                  (input logic [N-1:0] D,
3                   input logic [2:0] S,
4                   output logic [N-1:0] Y);
5
6   always_comb
7     case (S)
8       3'b000: Y = D;                      // No hay corrimiento
9       3'b001: Y = { D[N-2:0], 1'b0 };     // SHL
10      3'b010: Y = { D[N-1], D[N-1:1] };   // SHR
11      3'b011: Y = 4'b0;                   // Bus=0
12      3'b100: Y = D;                      // No hay corrimiento
13      3'b101: Y = { D[N-2:0], D[N-1] };   // ROL
14      3'b110: Y = { D[0], D[N-1:1] };     // ROR
15      3'b111: Y = D;                      // No hay corrimiento
16    endcase
17 endmodule
```

Listing 5.3: Implementación de la unidad de corrimiento (`shifter.sv`).

Módulo del banco de registros (ram3port.sv)

```
module ram3port #(parameter M = 3, N = 4)
                 (input logic clk,
                  input logic we3,
                  input logic [M-1:0] a1, a2, a3,
                  output logic [N-1:0] d1, d2,
                  input logic [N-1:0] d3);

  logic [N-1:0] mem[2**M-1:0];

  always_ff @(posedge clk)
    if (we3) mem[a3] <= d3;

  assign d1 = mem[a1];
  assign d2 = mem[a2];
endmodule
```

Listing 5.4: Banco de registros de tres puertos (`ram3port.sv`).

Banco de pruebas (up_harvard_tb.sv)

```
`timescale 1ns/1ps

module up_harvard_tb();
  logic clk;
  logic [13:0] control;
  logic [3:0] constin;
  logic [3:0] datain;
  logic [3:0] dataout, addressout;
  logic [3:0] flags;
  logic [21:0] testvector [7:0];
  integer i;

  up_harvard dut (
    .clk(clk),
    .dIN(datain),
    .cIN(constin),
    .regCTRL(control),
    .dOUT(dataout),
    .aOUT(addressout),
    .sOUT(flags)
  );

  initial begin
    $dumpfile("ondas.vcd");
    $dumpvars(0, up_harvard_tb);
    $readmemb("../tb/up_harvard_mem.txt", testvector);
    for(i=0; i<8; i=i+1) begin
      {control, constin, datain} = testvector[i];
      clk = 0;
      #10;
      clk = 1;
      #10;
    end
  end
endmodule
```

Listing 5.5: Banco de pruebas para verificación funcional (`up_harvard_tb.sv`).

Archivo de vectores de prueba (up_harvard_mem.txt)

```
00_00_00_1_0_1_00000_0000_0101 // data addr 0, load en, data in en, datain=5 (R0 = 5)
01_00_00_1_0_1_00000_0000_0011 // data addr 1, load en, data in en, datain=3 (R1 = 3)
10_01_00_1_0_0_10010_0000_0000 // data addr 2, a addr 0, b addr 1, load en, alu en, SUM (R2
    = R0 + R1)
11_10_11_0_0_0_00000_0000_0000 // data addr 3 (ro), a addr 3, b addr 2 (DataOut = R2,
    AddrOut = R3)
10_11_10_1_1_0_10101_0010_0000 // data addr 2, a addr 2, b addr 3 (ign), load en, const en,
    const=2, alu en, SUB (R2 = R2 + ~const  + 1)
11_10_11_0_0_0_00000_0000_0000 // data addr 3 (ro), a addr 3, b addr 2 (DataOut = R2,
    AddrOut = R3)
10_10_10_1_0_0_00001_0000_0000 // data addr 2, a addr 2, b addr 2, load en, shifter en, SHL
    (R2 = R2 << 1)
11_10_11_0_0_0_00000_0000_0000 // data addr 3 (ro), a addr 3, b addr 2 (DataOut = R2,
    AddrOut = R3)
```

Listing 5.6: Patrones de prueba para el banco de pruebas (`up_harvard_mem.txt`).